



## **DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

### **Design and Analysis of Algorithms (CSE533)**

**B. Tech – Computer Science and Engineering**

**By:**

S M Tejashree Kashyap 2460440

Swathi P 2460478

Suraj Adhikari 2460457

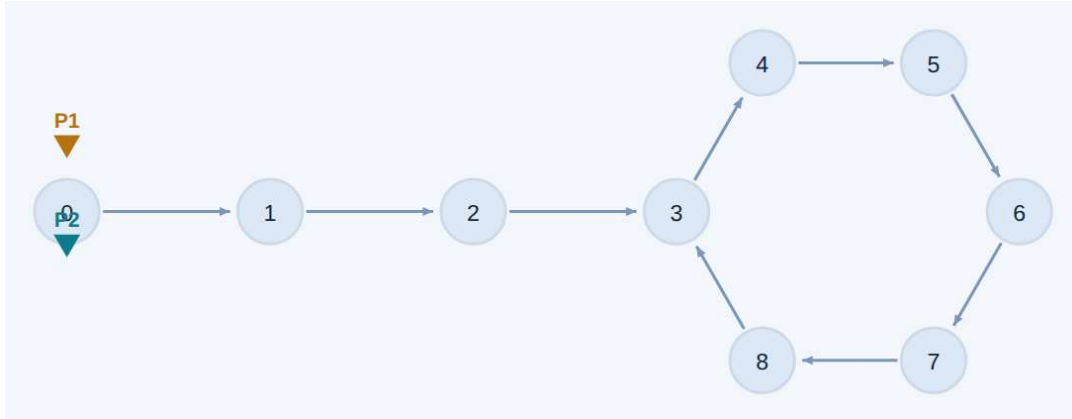
**School of Engineering and Technology, CHRIST  
(Deemed to be University), Kumbalagodu,  
Bengaluru - 560 074.**

September 2026

# Floyd's Cycle Detection Algorithm

*(Floyd's Cycle-Finding Algorithm)*

## Step-by-Step Worked Solution



*The example linked list used throughout this solution, rendered by the accompanying simulator.*

*Two pointers — a slow pointer P1 and a fast pointer P2 — traverse a singly linked list to detect whether it contains a cycle, and if so, to locate the exact node where the cycle begins, in  $O(n)$  time and  $O(1)$  extra space.*

# 1. Problem Statement

You are given the head of a singly linked list. Each node stores a value and a pointer to the next node. The last node's pointer may either be null (a straight list) or point back to some earlier node in the same list, creating a cycle.

Design an algorithm that:

- Determines whether the linked list contains a cycle.
- If a cycle exists, identifies the exact node at which the cycle begins.
- Uses only  $O(1)$  extra memory (no hash sets or visited-node marking) and  $O(n)$  time.

Floyd's Cycle Detection Algorithm (also called the "two-pointer" or "fast and slow pointer" technique) solves both requirements in a single linear pass followed by a second bounded pass, without any extra data structure.

## 2. Algorithm Idea

Two pointers start at the head of the list:

- P1 (slow pointer) advances one node per iteration.
- P2 (fast pointer) advances two nodes per iteration.

If the list has no cycle, P2 reaches the end (a null pointer) first, and the algorithm reports that no cycle exists.

If the list does contain a cycle, P2 enters the cycle before P1 and begins "lapping" it from behind. Because P2 gains exactly one extra node of distance on P1 every iteration, and the cycle has finite length, P2 is guaranteed to catch up to and land exactly on P1 within at most one full lap of the cycle. The node where they land on each other is called the meeting node.

Detecting the meeting node proves a cycle exists, but the meeting node is generally not the start of the cycle. A second phase is required to locate it:

- Reset a new pointer to the head of the list; leave P1 at the meeting node.
- Advance both pointers one node at a time.
- The node at which they meet is provably the first node of the cycle. (Proved in Section 6.)

## 3. Pseudocode

### Phase 1 — Detect a cycle

```
P1 ← head
P2 ← head
while P2 ≠ null and P2.next ≠ null:
    P1 ← P1.next
    P2 ← P2.next.next
    if P1 == P2:
        break           // cycle detected, P1 == P2 = meeting node
if P2 == null or P2.next == null:
    return "no cycle found"
```

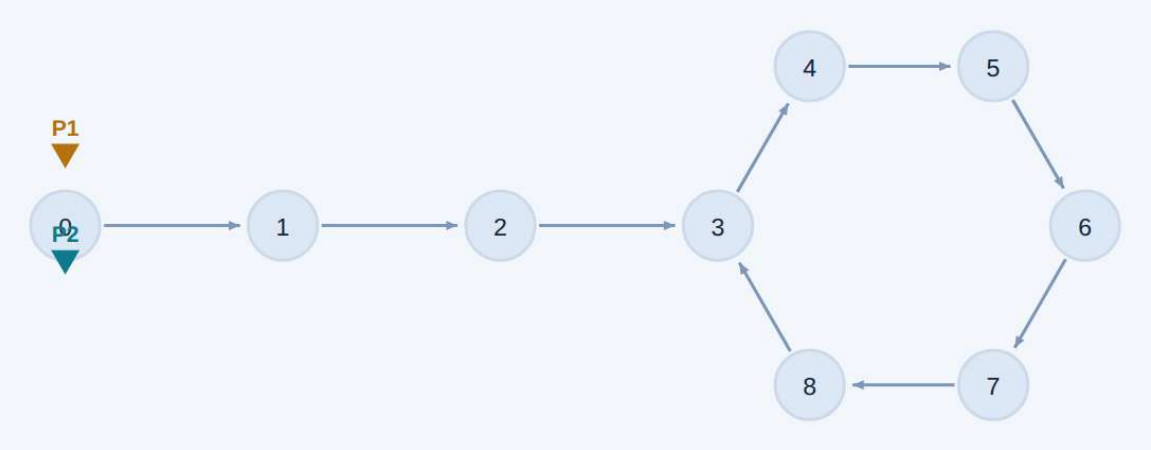
### Phase 2 — Locate the cycle's start

```
ptr ← head
```

```
while ptr ≠ P1:
    ptr ← ptr.next
    P1 ← P1.next
return ptr          // first node of the cycle
```

## 4. Worked Example

For this walkthrough the simulator was configured with  $N = 9$  nodes (indexed 0–8), with the last node's next pointer set to reconnect at index 3. This produces a linear tail of length  $\mu = 3$  (nodes 0, 1, 2) followed by a cycle of length  $\lambda = 6$  (nodes 3, 4, 5, 6, 7, 8, back to 3).



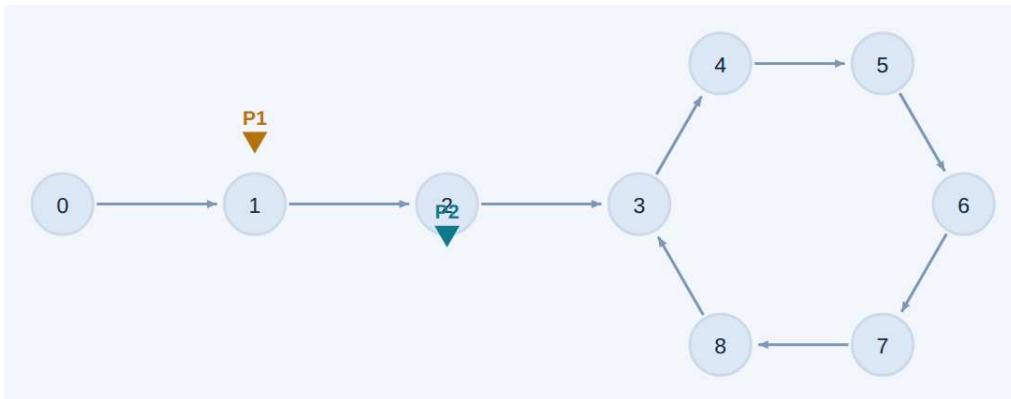
*Initial state: both P1 and P2 start at node 0 (the head).*

|                            |               |
|----------------------------|---------------|
| Tail length ( $\mu$ )      | <b>3</b>      |
| Cycle length ( $\lambda$ ) | <b>6</b>      |
| Actual cycle start         | <b>node 3</b> |

## 5. Phase 1 — Detecting the Cycle, Traced Step by Step

Each iteration below moves P1 by one node and P2 by two nodes, then checks whether they now occupy the same node.

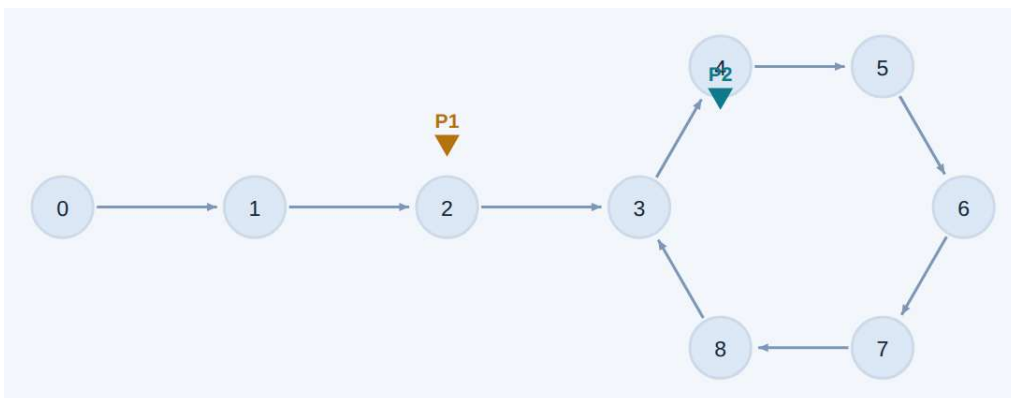
### Step 1 — Phase 1 (Detect)



Step 1. P1:  $0 \rightarrow 1$ . P2:  $0 \rightarrow 1 \rightarrow 2$ . Not equal yet — continue.

|           |                            |
|-----------|----------------------------|
| P1 (slow) | node 1 (1 step/iteration)  |
| P2 (fast) | node 2 (2 steps/iteration) |

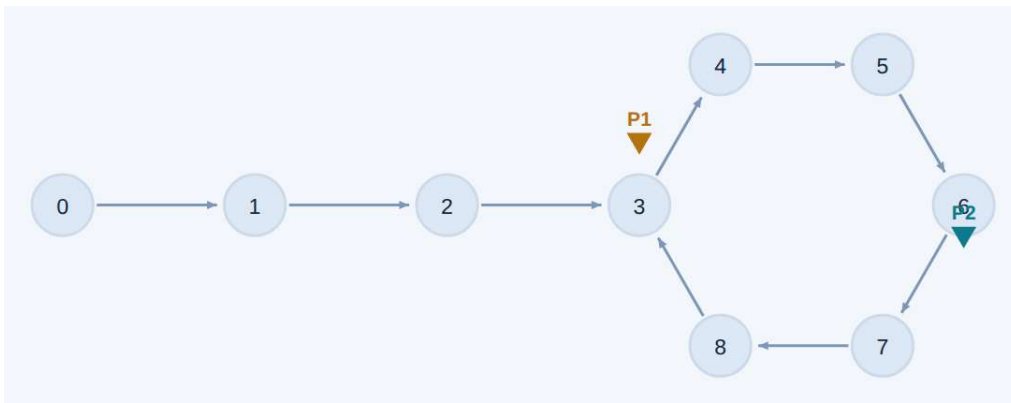
### Step 2 — Phase 1 (Detect)



Step 2. P1:  $1 \rightarrow 2$ . P2:  $2 \rightarrow 3 \rightarrow 4$ . Not equal yet — continue.

|           |                            |
|-----------|----------------------------|
| P1 (slow) | node 2 (1 step/iteration)  |
| P2 (fast) | node 4 (2 steps/iteration) |

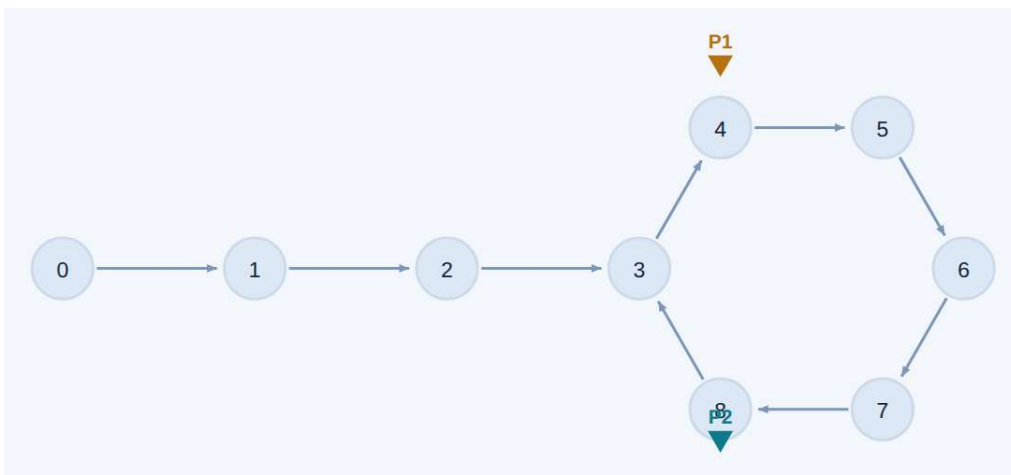
### Step 3 — Phase 1 (Detect)



Step 3. P1: 2 → 3. P2: 4 → 5 → 6. Not equal yet — continue.

|           |                                   |
|-----------|-----------------------------------|
| P1 (slow) | <b>node 3 (1 step/iteration)</b>  |
| P2 (fast) | <b>node 6 (2 steps/iteration)</b> |

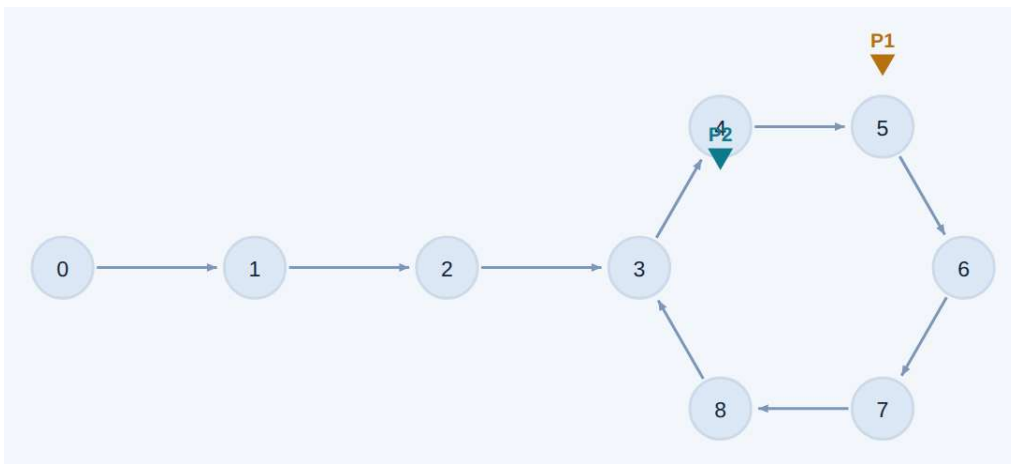
#### Step 4 — Phase 1 (Detect)



Step 4. P1: 3 → 4. P2: 6 → 7 → 8. Not equal yet — continue.

|           |                                   |
|-----------|-----------------------------------|
| P1 (slow) | <b>node 4 (1 step/iteration)</b>  |
| P2 (fast) | <b>node 8 (2 steps/iteration)</b> |

#### Step 5 — Phase 1 (Detect)



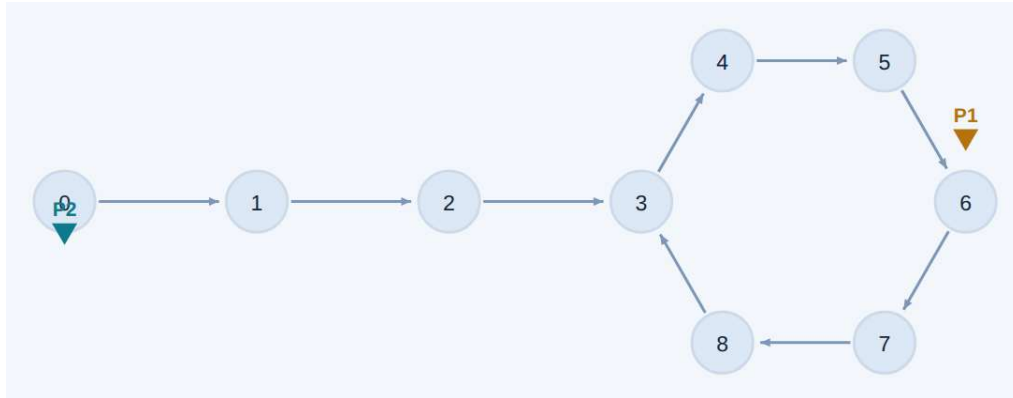
Step 5. P1: 4 → 5. P2: 8 → 3 → 4. Not equal yet — continue.

|           |                                   |
|-----------|-----------------------------------|
| P1 (slow) | <b>node 5 (1 step/iteration)</b>  |
| P2 (fast) | <b>node 4 (2 steps/iteration)</b> |

## 6. Phase 2 — Locating the Start of the Cycle

A cycle has now been confirmed at the meeting node found above. To find where the cycle begins, a fresh pointer starts again from the head while P1 stays at the meeting node. Both now advance one node at a time.

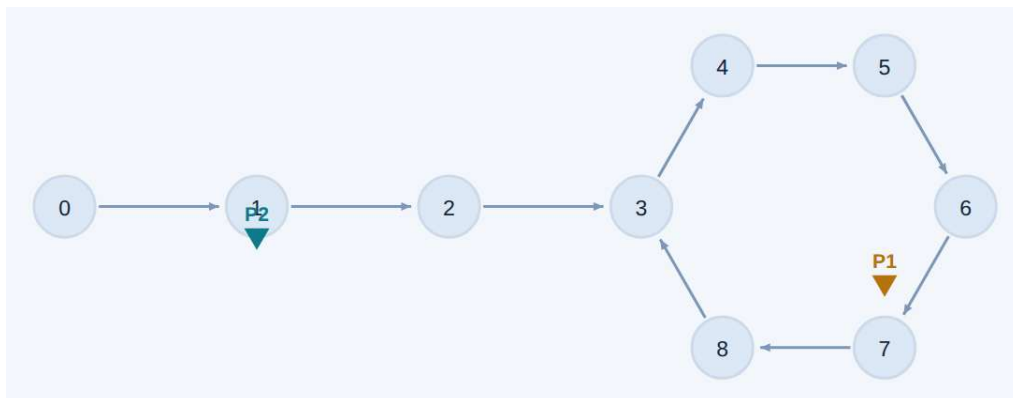
### Step 6 — Phase 2 (Locate)



Step 6. P1: 5 → 6. P2: 4 → 5 → 6. Both land on the same node — cycle confirmed at node 6. Phase 2 begins: find where the cycle starts.

|                     |                             |
|---------------------|-----------------------------|
| P1                  | node 6 (from meeting point) |
| Head-search pointer | node 0 (from head)          |

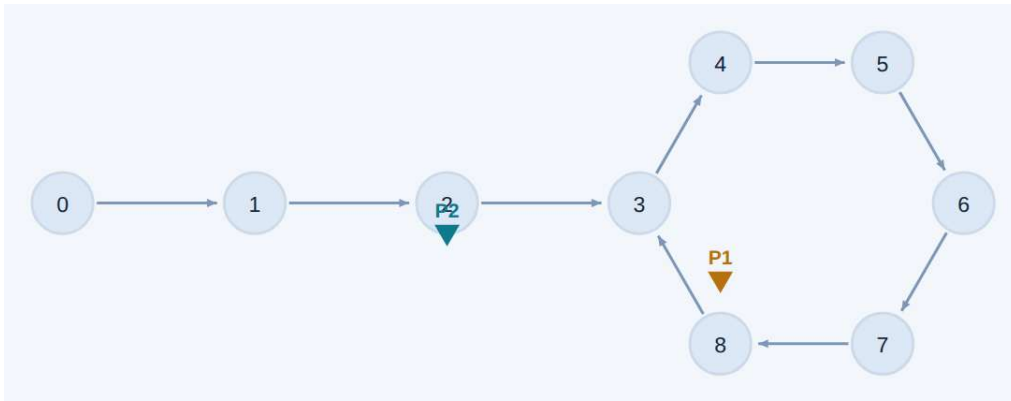
### Step 7 — Phase 2 (Locate)



Step 7. Head-pointer: 0 → 1. Meeting-pointer: 6 → 7. Not yet aligned — both advance one node at a time.

|                     |                             |
|---------------------|-----------------------------|
| P1                  | node 7 (from meeting point) |
| Head-search pointer | node 1 (from head)          |

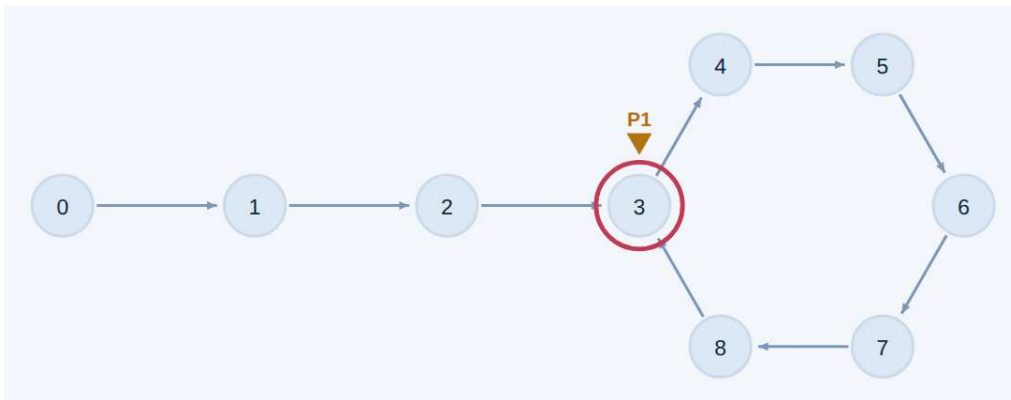
### Step 8 — Phase 2 (Locate)



Step 8. Head-pointer:  $1 \rightarrow 2$ . Meeting-pointer:  $7 \rightarrow 8$ . Not yet aligned — both advance one node at a time.

|                     |                                    |
|---------------------|------------------------------------|
| P1                  | <b>node 8 (from meeting point)</b> |
| Head-search pointer | <b>node 2 (from head)</b>          |

### Step 9 — Result



Step 9. Head-pointer:  $2 \rightarrow 3$ . Meeting-pointer:  $8 \rightarrow 3$ . They meet — node 3 is the start of the cycle.

|                     |               |
|---------------------|---------------|
| P1                  | <b>node 3</b> |
| Head-search pointer | <b>node 3</b> |

## 7. Result

Floyd's algorithm reports that the linked list contains a cycle, and that the cycle begins at node 3.

✓ This matches the actual cycle start built into the example (node 3), confirming the algorithm's correctness on this input.

## 8. Why the Algorithm Works — the Underlying Mathematics

Let  $\mu$  be the length of the tail before the cycle (the number of non-cycle nodes reachable from the head), and let  $\lambda$  be the length of the cycle itself.

### 8.1 Phase 1 always terminates with $P1 == P2$ inside the cycle

Once P1 enters the cycle, P2 is already inside it (P2 always leads P1). From that point on, each iteration reduces the gap between P2 and P1, measured going forward around the cycle, by exactly one node — because P2 gains two steps while P1 gains one. A gap that shrinks by 1 every iteration, modulo a cycle of finite length  $\lambda$ , must reach 0 within at most  $\lambda$  iterations. When the gap is 0, P1 and P2 occupy the same node: they have met.

### 8.2 Distance relationship at the meeting point

Suppose the meeting happens after P1 has taken  $k$  steps from the head (so P1 has walked  $k = \mu + j$  nodes for some  $j \geq 0$  inside the cycle). P2 has taken  $2k$  steps in the same time, and both are on the same node, so:

$$\begin{aligned} 2k &\equiv k \pmod{\lambda} && \text{[same node reached by both, difference is a multiple of } \lambda \text{]} \\ \Rightarrow k &\equiv 0 \pmod{\lambda} \\ \Rightarrow k &= m \cdot \lambda \text{ for some integer } m \geq 1 \end{aligned}$$

Since  $k = \mu + j$ , this means  $\mu + j = m \cdot \lambda$ , i.e.  $j = m \cdot \lambda - \mu$ . In other words, the number of extra steps still needed to walk from the meeting node all the way around the cycle back to the start (that remaining distance is  $\lambda - j$ ) is exactly congruent to  $\mu$ , modulo  $\lambda$ :

$$\lambda - j \equiv \mu \pmod{\lambda}$$

### 8.3 Why Phase 2 finds the exact start

Phase 2 places one pointer at the head (distance  $\mu$  from the cycle start) and leaves the other at the meeting node (distance  $\lambda - j$  from the cycle start, going forward around the cycle — and by the relation above, this is congruent to  $\mu$  modulo  $\lambda$ ). Advancing both pointers one node at a time, the head-pointer reaches the cycle start after exactly  $\mu$  steps; the meeting-point pointer, having an equivalent distance modulo the cycle length, arrives at the same node at the same time. Hence the two pointers are guaranteed to meet precisely at the first node of the cycle — never earlier, never later.

For this worked example,  $\mu = 3$  and  $\lambda = 6$ . The meeting node found in Phase 1 was node 6, and Phase 2 needed exactly 3 further steps (matching  $\mu$ ) before both pointers arrived together at node 3 — the true start of the cycle.

## 9. Complexity Analysis

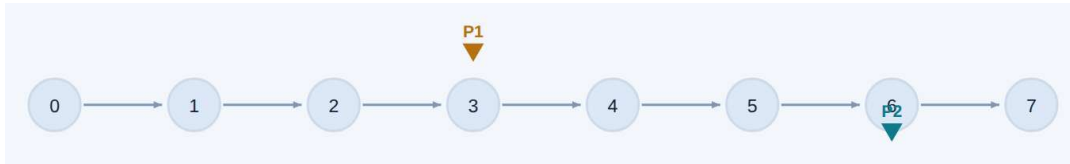
|                  |                                                                                                                      |
|------------------|----------------------------------------------------------------------------------------------------------------------|
| Time complexity  | <b><math>O(n)</math> — each pointer visits at most <math>O(\mu + \lambda) = O(n)</math> nodes across both phases</b> |
| Space complexity | <b><math>O(1)</math> — only two (later three) pointers are ever stored, regardless of list size</b>                  |

This compares favourably with a hash-set based approach, which also runs in  $O(n)$  time but requires  $O(n)$  extra space to remember every node already visited.

## 10. Additional Cases Verified with the Simulator

### 10.1 No cycle

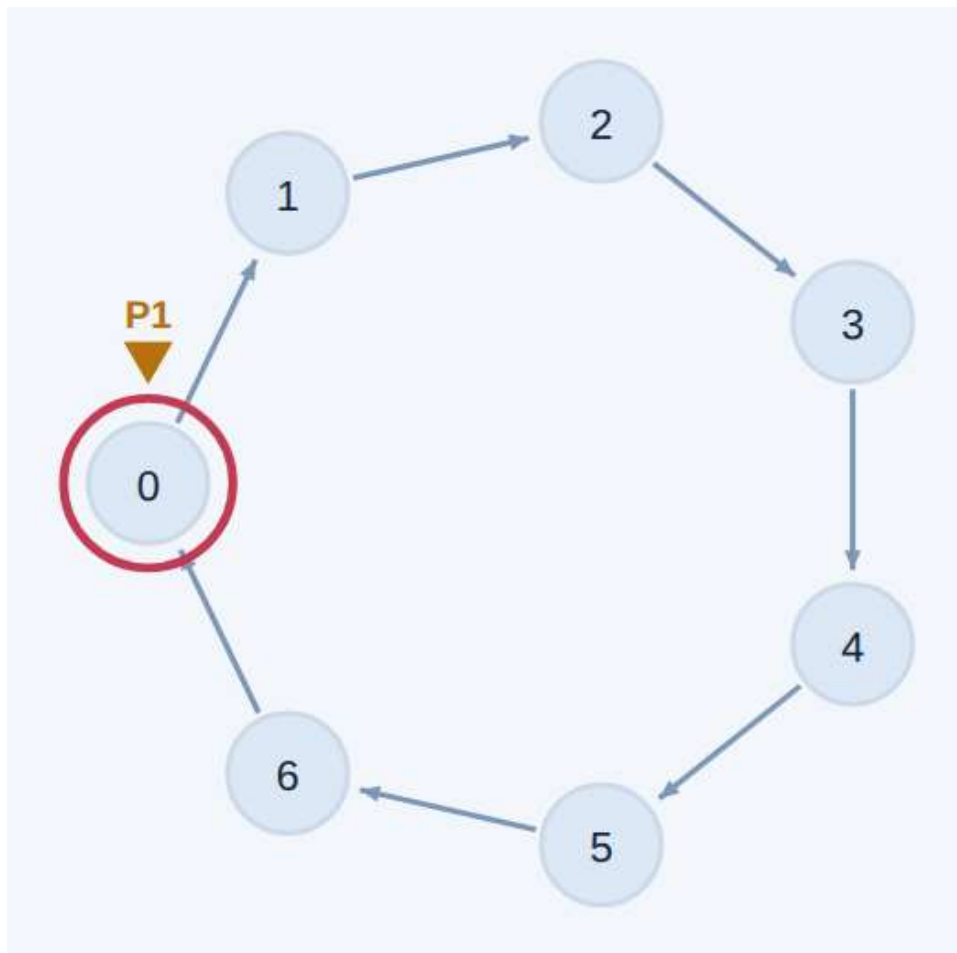
With  $N = 8$  nodes and no back-pointer ( $\text{pos} = -1$ ), P2 simply runs off the end of the list. Phase 1 terminates via the null check rather than a pointer match, and the algorithm correctly reports no cycle.



*P2 falls off the end at node 7  $\rightarrow$  null; P1 is left at node 3. No cycle exists.*

### 10.2 The entire list is a cycle ( $\mu = 0$ )

With  $N = 7$  nodes and  $\text{pos} = 0$ , the last node connects back to the head itself, so every node is part of the cycle and the tail length is 0. The algorithm still correctly identifies node 0 as the cycle start, consistent with  $\mu = 0$  in the formula derived in Section 8.



*When the whole list is one cycle, Floyd's algorithm reports the head itself as the cycle start.*

## 11. Conclusion

Floyd's Cycle Detection Algorithm reliably detects the presence of a cycle in a singly linked list and pinpoints its starting node, using only two (briefly three) pointers,  $O(n)$  time, and  $O(1)$  additional space. The step-by-step trace above — generated directly from the accompanying interactive simulator — demonstrates both phases of the algorithm on a concrete example, and the underlying modular-arithmetic argument in Section 8 explains why the second phase is guaranteed to land exactly on the first node of the cycle.